Results in Control and Optimization 4 (2021) 100048

Contents lists available at ScienceDirect

Results in Control and Optimization

journal homepage: www.elsevier.com/locate/rico

Reinforcement learning with constraint based on mirror descent algorithm

Megumi Miyashita [a,*], Toshiyuki Kondo [b], Shiro Yano [b]

[a] Department of Electronic and Information Engineering, Graduate School of Engineering, Tokyo University of Agriculture and Technology, 2-24-16 Naka-cho, Koganei-shi, Tokyo, Japan

[b] Division of Advanced Information Technology and Computer Science, Institute of Engineering, Tokyo University of Agriculture and Technology, 2-24-16 Naka-cho, Koganei-shi, Tokyo, Japan

ARTICLE INFO

Keywords:
Reinforcement learning
Constrained optimization
Mirror descent algorithm

ABSTRACT

An important issue in reinforcement learning is to make the agent avoid the dangers and risks during the task such as physical collisions. We propose the reinforcement learning algorithm based on the CoMirror algorithm, named CoMDS, for the problem that has a functional constraint. Besides, we modify the proposed algorithm CoMDS to Gaussian CoMDS for practical use. We evaluate our algorithms with the via-point task of a planar robotic arm with a forbidden area, that employs as a constraint, in the simulator. As a result, we find that Gaussian CoMDS explores the policy while satisfying the constraint.

1. Introduction

An agent in reinforcement learning (RL) learns the skills by trial-and-error. The agent interacts with the environment by executing the action generated by the policy. The environment returns a reward as feedback of given action, and transitions to the next state. Then, the agent executes the next action for the environment. To obtain the desired skills, the agent explores the policy to maximize the reward.

An important issue in this field is to make the agent avoid the dangers and risks such as physical collisions during the task. To deal with danger, RL that introduced the safety concept has been studied [1]. That is called the safe RL. The safe RL methods can be divided into (1) a transformation of the optimization criterion, and (2) a modification of the exploration process [1]. As the modification of the optimization criterion affects the modification of the exploration process, they cannot be completely isolated.

We focus on the formulation based on the constrained optimization problem. In the context of RL and constraint, the constrained Markov decision process (CMDP) [2] has been studied since 1999. This is the extension of the Markov decision process (MDP). In these studies, the MDP environment is assumed, so the next state and the reward depend on only the current state and the given action. The difference between MDP and CMDP is that the CMDP environment returns not only the reward but also the cost. Most RL algorithm is based on the assumption that the environment can be described as the MDP. It is because the MDP assumption makes the algorithms simpler, however, the environment does not always satisfy the MDP. Now, we focus on the algorithm for constraint settings not depending on the MDP assumption.

We propose CoMirror descent search (CoMDS), which is the reinforcement learning algorithm based on the CoMirror algorithm [3]. The CoMirror algorithm is the extension of the mirror descent algorithm, which is a gradient-based optimization algorithm. The advantages of the proposed method are (1) providing the possibility to apply the constraint to the existing RL algorithm based on the mirror descent algorithm; (2) considering the constraint with a simple implementation; and (3) applying the environment without MDP assumption. We apply our algorithm to the simple robotic task with constraints in a simulator.

* Corresponding author.
E-mail address: miyashita@livingsyslab.org (M. Miyashita).

https://doi.org/10.1016/j.rico.2021.100048
Received 18 May 2021; Received in revised form 29 June 2021; Accepted 14 August 2021
Available online 20 August 2021
2666-7207/© 2021 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

2. Related work

We summarize the safe RL research from some important viewpoints.

A part of the safe RL algorithms employs the techniques of constrained optimization algorithms. Projected natural actor critics (PNACs) [4] are RL algorithms based on the natural actor–critic for the problem with constraints. PNACs project the policy onto the given safe policies. The past research proposed the RL algorithms for the constrained RL problem based on the primal–dual method [5,6]. Constrained policy optimization (CPO) [7] is an algorithm based on the trust region method. CPO is motivated by the trust region policy optimization (TRPO) [8] that succeeds in the setting with high-dimensional parameters. Approachability-based policy optimization (ApproPO) [9] is the algorithm based on the knowledge of game theory and online convex optimization. Reward Constrained Policy Optimization (RCPO) [9] is the technique to deal with the problem with constraint by reshaping the reward. They need MDP structure explicitly, on the other hand, the proposed algorithm need not assume MDP structure.

An increasing number of studies propose RL algorithms based on the mirror descent algorithm. Mirror Decent Guided Policy Search (MDGPS) [10] provides the interpretation of Guided Policy Search (GPS) [11] as an approximate variant of the mirror descent algorithm. GPS is the algorithm to optimize the policy in the high-dimensional parameter space with the technique of supervised learning. Exploratory Conservative Policy Optimization (ECPO) [12] is an algorithm based on a mirror descent algorithm by focusing on an entropy and relative entropy regularizer. Divergence-Augmented Policy Optimization (DAPO) [13] is an algorithm focusing on a Bregman divergence for data efficiency. The mirror descent algorithm uses a Bregman divergence like a metric. They also provide a discussion between policy optimization and mirror descent algorithm. In this research, we apply our technique to the RL algorithms based on mirror descent. Moreover, the mirror descent algorithm has a relationship with many optimization algorithms, because of the generality of the mirror descent algorithm. Some algorithms such as TRPO and the relative entropy policy search (REPS) [14] have the relationship to the mirror descent [15,16]. Our technique is potentially applicable to these methods.

Some works in the field of optimization are affected by CoMirror algorithm [3]. First, Bauschke et al. [17] proposed the derivative-free CoMirror algorithm. They provided the analysis of convergence under the assumption that each function is lower-$\mathcal{C}^2$. This functional assumption provides a method to approximate subgradients. The motivation for the CoMirror algorithm to extend the derivative-free is similar to our work. However, the work is quite different from our work in terms of the approach. Second, Titov et al. [18] proposed the algorithm based on the mirror descent algorithm to the problem with the constraint. In that work, they mainly discussed adaptive stepsize. The difference from our work is that the two works did not have a relationship to RL.

3. Preliminaries

First, we introduce a formulation of the optimization problem and the constrained RL problem. Then we summarize the relationship between them.

3.1. Optimization problem and constraint setting

The maximization problem can be written as:

$$w^* = \arg\max_{w \in \mathcal{W}} \quad f(w), \tag{1}$$

where $f(w)$ is an objective function, $w \in \mathcal{W}$ a decision variable, and w^* an optimal decision variable.

Sometimes the decision variable w is limited to not only the constraint $\mathcal{W}$ but also the additional constraint. The maximization problem with a constraint function $g(w)$ can be written as:

$$\begin{aligned} w^* = \arg\max_{w \in \mathcal{W}} \quad & f(w) \\ \text{subject to} \quad & g(w) \leq 0. \end{aligned} \tag{2}$$

3.2. Reinforcement learning and constraint setting

In RL, an agent aims to find a better policy to maximize the expected reward. A policy $\pi : \mathcal{S} \to \mathcal{A}$ maps a state $\boldsymbol{s} \in \mathcal{S}$ to an action $\boldsymbol{a} \in \mathcal{A}$. The environment transitions the next state $\boldsymbol{s}_{t+1}$ according to the transition function $p\left(\boldsymbol{s}_{t+1} \mid \boldsymbol{s}_t, \boldsymbol{a}_t\right)$, the current state $\boldsymbol{s}_t$, and the given action $\boldsymbol{a}_t$. The temporal sequence of the state $\boldsymbol{s}$ and the action $\boldsymbol{a}$ is called the trajectory $\boldsymbol{\xi} := \left(\boldsymbol{s}_0, \boldsymbol{a}_0, \boldsymbol{s}_1, \boldsymbol{a}_1 \ldots\right)$. Then, the environment returns the value of the reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. The agent needs to explore the policy by trial-and-error because the agent does not have the knowledge about the reward function and the transition function in advance.

In a constrained setting, the environment also returns the value of the cost function $d : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ as with the reward. The constraint is satisfied when the cost value is lower than the given threshold. The agent needs to explore the policy while satisfying the constraint.

3.3. Black-box optimization problem and reinforcement learning

In some optimization problems, the form of the objective function or the constraint function is unknown. The optimization problem that has an unknown function is called a black-box optimization (BBO) problem. In such a problem, the analytical gradient of the function is unavailable. Thus, we usually use derivative-free/gradient-free optimization methods to solve the BBO problem. In those methods, gradients are unreliable or unavailable.

In RL, the agent aims to find a better policy to maximize the expected reward, as stated previously. Connecting BBO and RL, we can consider the objective function as the reward function, the decision variable as the policy, and the constraint function as the cost function. RL algorithm based on the view of BBO is in the class in the episode-based RL algorithm [19]. The episode-based RL algorithm usually explores and evaluates the policy for each trajectory. Typically, an episode-based RL algorithm uses the policy π that are parameterized by the policy parameter $\boldsymbol{\theta} \in \Theta$ [20]. In this paper, we assume the algorithm to use the parameterized policy.

Now, we illustrate the detail of the policy function. Before describing the policy in the episode-based RL algorithm, we introduce the class of the step-based RL algorithm, which is contrasting concepts for episode-based. The step-based RL algorithm usually explores and evaluates the policy for each state. The step-based RL algorithm can be applied both for a stochastic policy $\pi = p(\boldsymbol{a} \mid s, \boldsymbol{\theta})$ and a deterministic one $\pi(s, \boldsymbol{\theta})$. For the following description, we use the stochastic policy representation because the deterministic policy can be considered as the special case of the stochastic policy. In episode-based RL algorithm, the policy function corresponding to the action $p(\boldsymbol{a} \mid s, \boldsymbol{\theta})$ can be interpreted as that corresponding to the trajectory $p(\boldsymbol{\xi} \mid \boldsymbol{\theta})$ by considering the probabilistic transition function $p\left(\boldsymbol{s}_{t+1} \mid \boldsymbol{s}_t, \boldsymbol{a}_t\right)$. Hereinafter, this type of policy function is called the lower-level policy function. In addition, we can consider the upper-level policy $p(\boldsymbol{\theta})$ that is the distribution of the policy parameter. Note that the classification of the lower-level policy and the upper-level policy comes from [20].

The reward and the cost are also considered episodic. In step-based algorithm, the reward function is represented as $r(\boldsymbol{s}, \boldsymbol{a})$. In episode-based algorithm, the accumulated reward function $j(\boldsymbol{\xi}) = \sum_t r\left(\boldsymbol{s}_t, \boldsymbol{a}_t\right)$ is usually used. The expected reward function corresponding to the policy parameter $\boldsymbol{\theta}$ is:

$$J(\boldsymbol{\theta}) = \int_{\boldsymbol{\xi}} j(\boldsymbol{\xi})\, p(\boldsymbol{\xi} \mid \boldsymbol{\theta})\, \mathrm{d}\boldsymbol{\xi}. \tag{3}$$

In addition, the expected reward function corresponding to the upper-level policy function is:

$$\mathcal{J} = \int_{\boldsymbol{\theta}} p(\boldsymbol{\theta}) \int_{\boldsymbol{\xi}} j(\boldsymbol{\xi})\, p(\boldsymbol{\xi} \mid \boldsymbol{\theta})\, \mathrm{d}\boldsymbol{\xi}\, \mathrm{d}\boldsymbol{\theta} = \int_{\boldsymbol{\theta}} p(\boldsymbol{\theta})\, J(\boldsymbol{\theta})\, \mathrm{d}\boldsymbol{\theta}. \tag{4}$$

The agent archives its goal when it finds the policy to maximize $\mathcal{J}$.

If the agent is under the constrained setting, the agent needs to explore the policy while satisfying the constraint. Assuming that the cost is formulated the same procedure as the reward, the expected cost function is:

$$C = \int_{\boldsymbol{\theta}} p(\boldsymbol{\theta}) \int_{\boldsymbol{\xi}} c(\boldsymbol{\xi})\, p(\boldsymbol{\xi} \mid \boldsymbol{\theta})\, \mathrm{d}\boldsymbol{\xi}\, \mathrm{d}\boldsymbol{\theta} = \int_{\boldsymbol{\theta}} p(\boldsymbol{\theta})\, C(\boldsymbol{\theta})\, \mathrm{d}\boldsymbol{\theta}, \tag{5}$$

where $c(\boldsymbol{\xi})$ is the accumulated cost function $c(\boldsymbol{\xi}) = \sum_t d\left(\boldsymbol{s}_t, \boldsymbol{a}_t\right)$, and $C(\boldsymbol{\theta})$ is the expected cost function corresponding to the policy parameter $\boldsymbol{\theta}$.

4. CoMirror Descent Search

In this section, we propose an RL algorithm based on the CoMirror algorithm [3], which we term CoMirror Descent Search (CoMDS).

4.1. CoMirror algorithm

The original CoMirror algorithm is proposed as an extension of the mirror descent algorithm for convex problems with functional constraints by Beck A. et al. in 2010. In their work, they provided its convergence analysis and the experimental result. Some works are affected by their work [17,18].

The CoMirror algorithm for the constrained problem (2) is as follows:

$$\boldsymbol{w}_{k+1} = \arg\max_{\boldsymbol{w} \in \mathcal{W}} \left\{ \langle \boldsymbol{e}_k, \boldsymbol{w} \rangle - \eta B_\phi\left(\boldsymbol{w}, \boldsymbol{w}_k\right) \right\}, \tag{6}$$

$$\text{where } \boldsymbol{e}_k = \begin{cases} \nabla f(\boldsymbol{w}_k) & \text{if } g(\boldsymbol{w}_k) \le \epsilon \\ \nabla g(\boldsymbol{w}_k) & \text{otherwise,} \end{cases} \tag{7}$$

k is the index of iterations, η is the learning rate, and B_ϕ is Bregman divergence with function ϕ, ϵ is the threshold of switching the update direction. The gradient of the constraint function $\nabla g(\boldsymbol{w}_k)$ is used if the value of constraint $g(\boldsymbol{w}_k)$ exceeds the predefined threshold ϵ, the gradient of the objective function $\nabla f(\boldsymbol{w}_k)$ otherwise.

4.2. Applying CoMirror algorithm to reinforcement learning

Applying the CoMirror algorithm to the RL problem is not straightforward because we cannot differentiate the functions directly. Now, we assume that the vector $\boldsymbol{q}$ is corresponding to the policy distribution $p(\boldsymbol{\theta})$, whose domain is the simplex $\mathcal{Q}$. Based on the expected reward function in (4) and the expected cost function in (5), we can interpret the RL problem as follows:

$$\begin{aligned} \boldsymbol{q}^* = \underset{\boldsymbol{q}\in\mathcal{Q}}{\arg\max} \quad & \mathcal{J} \\ \text{subject to} \quad & C \leq 0. \end{aligned} \tag{8}$$

Then, the gradient of the functions can be calculated analytically as $\nabla_{\boldsymbol{q}}\mathcal{J} = \boldsymbol{J}$ and $\nabla_{\boldsymbol{q}} C = \boldsymbol{C}$. For more detail on this deformation, refer to [15].

The update equation is derived as:

$$\boldsymbol{q}_{k+1} = \underset{\boldsymbol{q}\in\mathcal{Q}}{\arg\max} \left\{ \langle \boldsymbol{E}_k, \boldsymbol{q} \rangle - \eta B_\phi \left(\boldsymbol{q}, \boldsymbol{q}_k \right) \right\}, \tag{9}$$

$$\text{where } \boldsymbol{E}_k = \begin{cases} \boldsymbol{J}_k & \text{if } \frac{1}{L}\sum_{l}^{L} \mathbb{1}\left[C_{k,l} \leq \epsilon\right] \geq \delta \\ \boldsymbol{C}_k & \text{otherwise,} \end{cases} \tag{10}$$

$\boldsymbol{J}_k$ and $\boldsymbol{C}_k$ is the values of the reward and the cost corresponding to the policy function $p_k(\boldsymbol{\theta})$, respectively. $l \in \{1, \dots, L\}$ is the index of samplings and $\delta \in [0, 1]$ is the threshold of the satisfy rate for samples. $\frac{1}{L}\sum_{l}^{L}\left[C_{k,l} \leq \epsilon\right] \geq \delta$ is the sample approximation of the chance constraint condition $p\left(\boldsymbol{C}_k \leq \epsilon\right) \geq \delta$. $\mathbb{1}$ in (10) is the indicator function. If we set δ to 0, CoMDS completely ignores the value of constraint. All procedures are shown in Algorithm 1. Note that we apply Monte Carlo approximation for (3) because $p(\boldsymbol{\xi} \mid \boldsymbol{\theta})$ is unknown before the iteration. It is as follows:

$$\boldsymbol{\xi}_m \sim p(\boldsymbol{\xi} \mid \boldsymbol{\theta}), \qquad J(\boldsymbol{\theta}) \simeq \frac{1}{M}\sum_{m=1}^{M} j\left(\boldsymbol{\xi}_m\right), \tag{11}$$

where $M \in \mathbb{N}$.

Algorithm 1 CoMirror Descent Search

initialize
  Policy distribution p_1, parameter ϵ, and $\delta \in [0, 1]$ arbitrarily.
for iteration $k \in \{1, \dots, K\}$ **do**
  for sampling policy $l \in \{1, \dots, L\}$ **do**
    Sample policy parameter $\boldsymbol{\theta}_l \sim p_k(\boldsymbol{\theta})$.
    for sampling trajectory $m \in \{1, \dots, M\}$ **do**
      Sample trajectory $\boldsymbol{\xi}_m \sim p(\boldsymbol{\xi} \mid \boldsymbol{\theta}_l)$.
    end for
    Evaluate policy parameter $J_{k,l} = \frac{1}{M}\sum_{\boldsymbol{\xi}} j(\boldsymbol{\xi})$ and $C_{k,l} = \frac{1}{M}\sum_{\boldsymbol{\xi}} c(\boldsymbol{\xi})$.
  end for
  Update policy distribution parameter with (9)
  Estimate p_{k+1} from $\boldsymbol{q}_{k+1}$
end for

To simplify implementation, we assume that the lower-level policy distribution follows the Gaussian distribution. Thus, we define the initial mean of the policy distribution $\boldsymbol{\mu}_1$ and the variance $\boldsymbol{\Sigma}$ in advance, instead of policy distribution itself. Moreover, by using the online mirror descent trick, we can approximate J to $j(\boldsymbol{\xi})$ for efficiency [15]. All procedures are shown in Algorithm 2.

Algorithm 2 Gaussian CoMirror Descent Search

initialize
  Mean of policy distribution $\boldsymbol{\mu}_1$, variance of policy distribution $\boldsymbol{\Sigma}$, parameter ϵ, and $\delta \in [0, 1]$ arbitrarily.
for iteration $k \in \{1, \dots, K\}$ **do**
  for sampling $l \in \{1, \dots, L\}$ **do**
    Sample policy parameter $\boldsymbol{\theta}_l \sim \mathcal{N}\left(\boldsymbol{\theta} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}\right)$.
    Sample trajectory $\boldsymbol{\xi} \sim p(\boldsymbol{\xi} \mid \boldsymbol{\theta}_l)$.
    Evaluate policy parameter $J_{k,l} = j(\boldsymbol{\xi})$, and $C_{k,l} = c(\boldsymbol{\xi})$.
  end for
  Update policy distribution with (9)
  Estimate the mean $\boldsymbol{\mu}_{k+1}$ from $\boldsymbol{q}_{k+1}$.
end for

Especially in cases of simulation environments, we can reduce the calculation time by running the sampling loop in parallel.

5. Experiments

We performed a via-point task of the 2DOF planar robotic arm with a constraint. This algorithm is suitable for the task assumed safety as constraint, thus we use constrained via-point task.

The agent obtains the higher reward when its end-effector passes through closer to a certain via-point at a certain time. The agent obtains the cost when its end-effector passes through the predefined forbidden area anytime. For example, the cost becomes 0 when the end-effector never passes through the forbidden area during the episode. The action $\boldsymbol{a} \in \mathbb{R}^2$ is the motor command for each joint. The state $\boldsymbol{s} \in \mathbb{R}^4$ contains the joint angular position $\tilde{\boldsymbol{v}} \in \mathbb{R}^2$ and the joint angular velocity $\tilde{\dot{\boldsymbol{v}}} \in \mathbb{R}^2$.

5.1. Experiment in different satisfy rate values

5.1.1. Policy representation

We used the Dynamic Movement Primitives (DMP) [21] as the representation of the policy $\pi(\boldsymbol{s}, \boldsymbol{\theta})$. Now, the policy parameter $\boldsymbol{\theta} \in \mathbb{R}^n$ for one-dimensional DMP is assumed in this description. The objective is to find each $\theta_i \in \mathbb{R}$ that produces better action. The movement is generated by the second-order dynamical system:

$$\ddot{v} = \tau^2 \alpha_v \left(\beta_v (G - v) - \dot{v}\right) + \tau^2 h, \tag{12}$$

where $v, \dot{v}, \ddot{v} \in \mathbb{R}$ correspond to the desired angular position, angular velocity and angular acceleration, respectively. τ is the time constant, $\alpha_v > 0$ and $\beta_v > 0$ are the predefined constants, and $G \in \mathbb{R}$ is a predefined goal state. h is the external force that defines the shape of the desired angular position trajectory as:

$$\ddot{x} = \tau^2 \alpha_x \left(\beta_x (G - x) - \dot{x}\right) \tag{13}$$

$$h(x) = \frac{\sum_i^n \psi_i(x) \theta_i x}{\sum_i^n \psi_i(x)}, \tag{14}$$

where $x \in \mathbb{R}$ is the phase variable, $\alpha_x > 0$, $\beta_x > 0$ are the predefined constants, and $\psi_i \in \mathbb{R}$ is the value of Gaussian basis function. For more detail on DMP, refer to [21].

In our experiment, we used the same dimensions of DMP as the DOF of the robotic arm. Then, the action $\boldsymbol{a}$ is calculated to follow the actual joint angular positions $\tilde{\boldsymbol{v}}$ to the desired joint angular positions $\boldsymbol{v}$ by the feedback control. Following equation shows the relationship by the feedback control, as follows:

$$a_i = \gamma_m \dot{v}_i + \gamma_d \tilde{\dot{v}}_i + K_p(v_i - \tilde{v}_i) + K_d(\dot{v}_i - \tilde{\dot{v}}_i), \quad i = 1, 2, \tag{15}$$

where $\gamma_m > 0$, $\gamma_d > 0$, $K_p > 0$, $K_d > 0$ are predefined constants, and i is the index of joints.

5.1.2. Parameter setting

As the experimental parameter, the number of iterations K is 2000, and the number of sampling L is 10. The dimension of DMP is 2 and the number of ψ for one-dimensional DMP, i.e. n, is 10, thus the number of policy parameters is 20. An initial trajectory from $(0, 0)$ to $(1, 1)$ is generated, then the agent obtains the reward proportional to the distance between its end-effector and the via-point $[0.5, 0.5]$ at 0.3s from the start. Constants are defined as follows: $\alpha_v = 25$, $\beta_v = 6.25$, $\alpha_x = 25$, $\beta_x = 6.25$, $\gamma_m = 1$, $\gamma_d = 1$, $K_p = 1$, and $K_d = 2$. For CoMDS, the parameter ϵ is 0 to detect any constraint violation.

5.1.3. Results

We compared Gaussian CoMDS under the condition $\delta = 0.0$, 0.5, and 1.0. As noted above, Gaussian CoMDS ($\delta = 0.0$) completely ignores the constraint condition, i.e., the forbidden area. We evaluated the methods from the viewpoints of the learning curve, the rate of satisfying, and the shape of the trajectory. The learning curve and the arm trajectory are shown in Fig. 1.

The blue lines in Fig. 1 show the results of Gaussian CoMDS ($\delta = 0.0$). While Gaussian CoMDS ($\delta = 0.0$) maximizes the reward straightforwardly (Fig. 1(a)), it completely ignores and violates the constraint conditions (Fig. 1(b) and Fig. 1(c)). The green lines show the results of Gaussian CoMDS ($\delta = 1.0$), and the orange lines show those of Gaussian CoMDS ($\delta = 0.5$). These methods gradually improve these policies (Fig. 1(a)) with satisfying the constraint conditions (Fig. 1(b) and Fig. 1(c)). Fig. 1(d) shows the trajectories of the policies optimized by Gaussian CoMDS ($\delta = \{0.0, 0.5, 1.0\}$). As the figure directly shows, the optimal policy of Gaussian CoMDS ($\delta = \{0.0, 0.5\}$) passes on the forbidden area, on the other hand, that of Gaussian CoMDS ($\delta = 1.0$) avoids the forbidden area. Fig. 1(a) shows that the standard deviation of reward in Gaussian CoMDS ($\delta = \{0.5, 1.0\}$) is high because the timing of updating policy parameter to the direction of maximizing reward are different depending on the seed.

5.2. Experiment in the environment with a refractory period

In this section, we introduce the environment with a *refractory period* as shown in Fig. 2. The refractory happens randomly. During the refractory period, the environment does not respond to any actions executed by the agent. However, the agent can observe

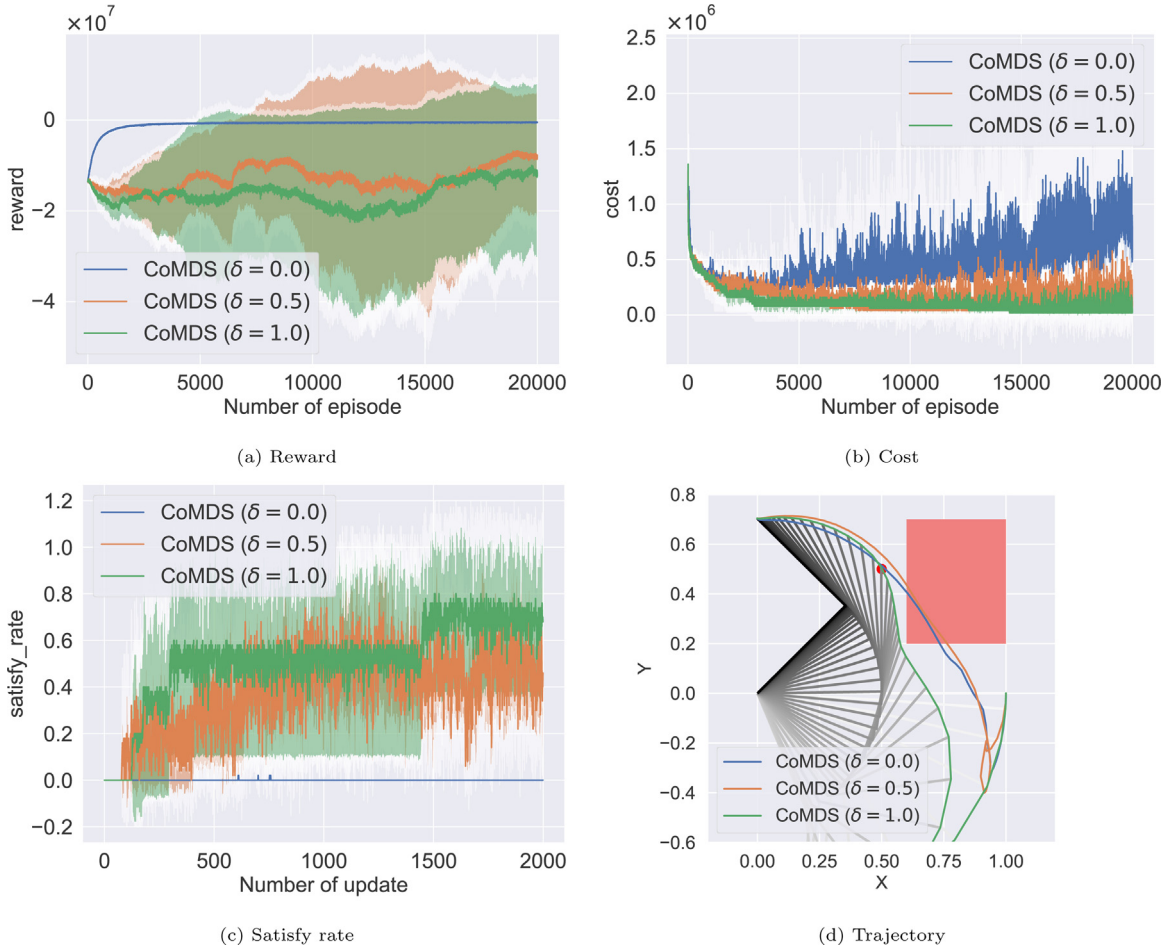


Fig. 1. Performance of Gaussian CoMDS ($\delta = 1.0$), ($\delta = 0.5$), and ($\delta = 0.0$) in a via-point task of the 2DOF planar robotic arm with a constraint. The green color shows Gaussian CoMDS ($\delta = 1.0$), the orange color shows Gaussian CoMDS ($\delta = 0.5$), and the blue color shows Gaussian CoMDS ($\delta = 0.0$). These experiments were executed 5-times in the different seeds. 1(a)–1(c) The solid line represents the average, and the light-colored area means the standard deviation. 1(d) The line shows the average trajectory at the last iteration. The red area is the predefined forbidden area. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

reward and state, which is the same as those before entering the refractory period. For example, the refractory period assumes the situation that the robot does not accept control command when its control network is frozen unexpectedly. Thus the refractory is worth thinking about because it often happens in many real-world. However, we need to be careful that the environment with the refractory period does not follow MDP.

We show that CoMDS can solve the task under the non-MDP in Section 5.2.2, and more the task with constrained settings in Section 5.2.3. Note that the behavior of CoMDS for the task with unconstrained setting is the same as one of MDS [15]. In addition, in unconstrained settings, the value of δ does not matter because it can be regarded as always satisfying the constraint.

In this experiment, the agent obtains the higher reward when its end-effector passes through closer to a certain via-point during a certain duration. We defined the reward duration in which the agent observes rewards at least once. We used a fully connected neural network with 2 hidden layers with 256 neurons respectively, as the representation of the policy.

5.2.1. Parameter setting

We set the number of iterations K is 2000, and the number of sampling L is 100. For the comparison between PG and CoMDS, the number of iteration is limited to be equivalent to the time steps 4.0×10^6. We compared them under the number of time steps because there are differences in the number of episodes using for one update. The simulation proceeds with a discretized time step of 0.01 s. In our problem, as noted in Fig. 2, the duration of the refractory period differs from each episode. The duration is uniformly chosen from $\{0.01\,\text{s}, 0.02\,\text{s}, 0.03\,\text{s}, 0.04\,\text{s}, 0.05\,\text{s}\}$.

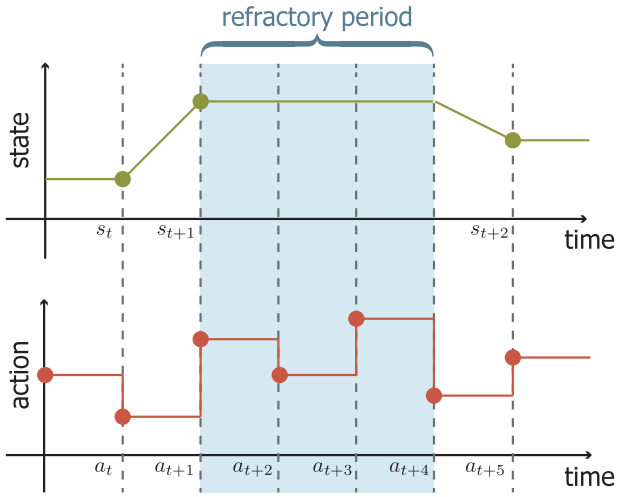
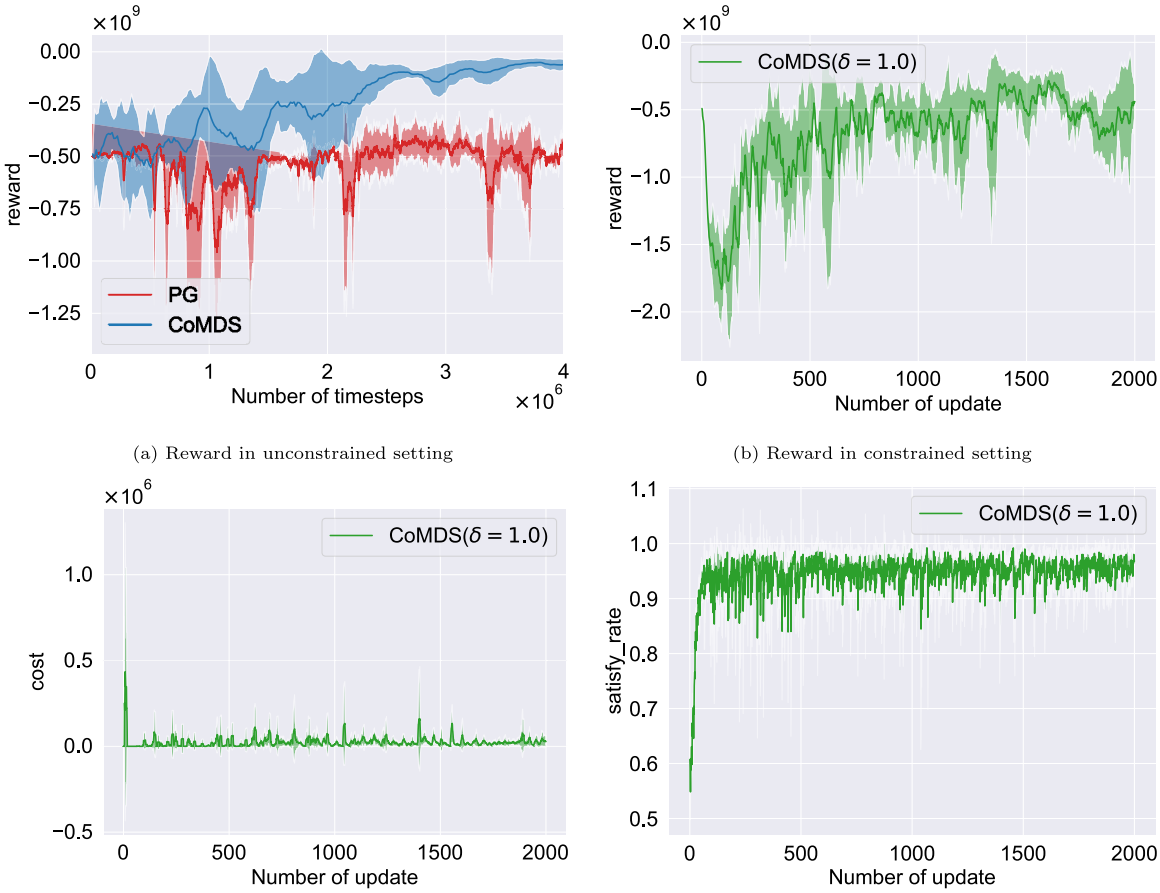


Fig. 2. The environment with a refractory period. A refractory period is illustrated by the period filled in blue. As the upper figure shows, the environment does not respond to any actions executed by the agent during the refractory period. In our problem setting, the refractory periods are mixed in the episode randomly. Moreover, the duration of the refractory period differs from each episode.



(a) Reward in unconstrained setting

(b) Reward in constrained setting

(c) Cost in constrained setting

(d) Satisfy rate in constained setting

Fig. 3. Performances of the RL algorithms in the environment with the refractory period. The green color shows Gaussian CoMDS ($\delta = 1.0$), the red color shows PG, and the blue color shows CoMDS (any δ). These experiments were executed 5-times in the different seeds. 3(a)–3(d) The solid line represents the average, and the light-colored area means the standard deviation. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

5.2.2. Result of unconstrained setting

We compared two RL algorithms by solving the task with the refractory period. One employs the MDP assumption and the other does not, i.e. Policy Gradient (PG) v.s. CoMDS. The learning curve is shown in Fig. 3(a). The result shows that the PG agent could not increase its cumulated reward under this setting. In contrast to PG, the CoMDS agent succeeded to increase its cumulated reward and improved policy function. The result indicates that the RL agents designed with the MDP assumption are vulnerable to non-MDP environments, especially refractory periods. It also indicates that the BBO agents hold robustness to the non-MDP environments.

5.2.3. Result of constrained setting

Even if the task includes both the constraint condition and the refractory period, Gaussian CoMDS ($\delta = 1.0$) agent solved the task successfully. The reward, cost and satisfy rate are shown in Fig. 3(b), Fig. 3(c), and Fig. 3(d), respectively. Between 0 to about 100 updates, while the agent could not increase the reward (Fig. 3(b)), the agent succeeded to decrease the cost and increase the satisfy rate (Fig. 3(c) and Fig. 3(d)). It seems that the CoMDS updated the policy parameter towards the direction of minimizing cost to deal with the low satisfy rate. After that, the agent updated the policy parameter to mainly the direction of maximizing reward as satisfy rate becomes large enough.

6. Conclusion

We proposed the RL algorithm for the problem that has a functional constraint based on the CoMirror algorithm [3]. We applied our algorithms, Gaussian CoMDS ($\delta = \{0.0, 0.5, 1.0\}$), to the simple robotic task with constraint in the simulator. Note that Gaussian CoMDS ($\delta = 0.0$) is the variation of the proposed algorithm that ignores the value of constraint. As a result, we found that Gaussian CoMDS ($\delta = 1.0$) explores the policy while satisfying the constraint.

The parameter δ can be tuned relying on the required satisfy rate. For example, when we need 50% of the satisfy rate at most, it is enough to set $\delta = 0.5$. It leads to the balance between maximizing a reward and satisfying a cost. Besides, the variants of the CoMirror algorithms are attractive, e.g., deciding the stepsize analytically provided in [18]. We hope to adopt this technique and provide an analysis of convergence to ensure performance.

The other existing algorithms are utilized high-dimensional parameters, such as deep neural network representation, and achieve an interesting result. We would work on evaluating our proposed algorithm with the higher dimensional parameters to discuss the scalability of the parameter dimension and to obtain new insights. Moreover, to identify the position of our proposed algorithm, it would be preferable to perform multiple tasks as an evaluation and some existing well-known algorithms would be added for comparison.

CoMDS can consider the constraint with a simple implementation and apply a task without assuming MDP. On the other hand, it is dangerous to solve a task where constraint violations are fatal because CoMDS cannot prevent constraint violations during searching. Moreover, CoMDS cannot apply to a task which not defined safety in advance.

Our work presented the general idea of dealing with constraints in RL based on the CoMirror algorithm. We will address the idea to improve the proposed method.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by JSPS KAKENHI Grant Number JP20J13766, JP19H05727, JP20H02111.

Appendix

Fig. 4 shows all trajectories of Gaussian CoMDS ($\delta = 0.5$) using DMPs as a policy representation. The trajectory of exp3 is out of the other trajectories. One experiment of $\delta = 0.5$ could not reach enough satisfy rate to update policy parameter to the direction of maximizing reward. Except for its trajectory, the trajectory of $\delta = 0.5$ is placed between $\delta = 0.0$ and $\delta = 1.0$.

Fig. 5 shows the learning curve of PG in the normal (without refractory period) environment. It shows PG can increase the reward in the normal environment. Thus PG can learn this task itself.

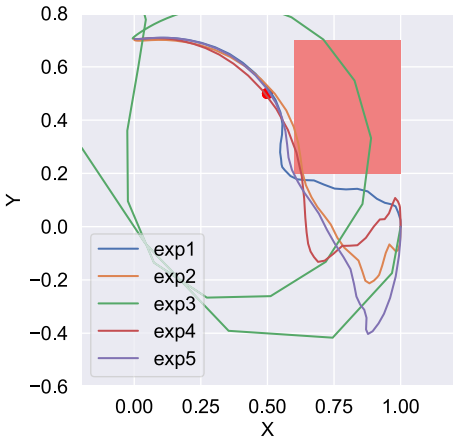


Fig. 4. All trajectories of Gaussian CoMDS ($\delta = 0.5$) using DMPs.

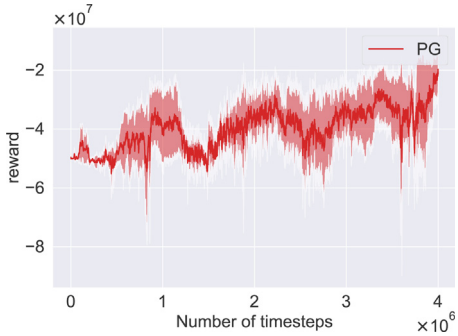


Fig. 5. PG Reward in the normal environment.

References

[1] Garcıa J, Fernández F. A comprehensive survey on safe reinforcement learning. J Mach Learn Res 2015;16(1):1437–80.

[2] Altman E. Constrained markov decision processes. vol. 7, CRC Press; 1999.

[3] Beck A, Ben-Tal A, Guttmann-Beck N, Tetruashvili L. The CoMirror algorithm for solving nonsmooth constrained convex problems. Oper Res Lett 2010;38(6):493–8. http://dx.doi.org/10.1016/j.orl.2010.08.005.

[4] Thomas PS, Dabney WC, Giguere S, Mahadevan S. Projected natural actor-critic. In: Advances in neural information processing systems. 2013, p. 2337–45.

[5] Liang Q, Que F, Modiano E. Accelerated primal-dual policy optimization for safe reinforcement learning. In: ArXiv E-Prints, arXiv:1802.06480. 2018, arXiv:1802.06480.

[6] Paternain S, Calvo-Fullana M, Chamon LFO, Ribeiro A. Safe policies for reinforcement learning via primal-dual methods. In: ArXiv E-Prints, arXiv:1911.09101. 2019, arXiv:1911.09101.

[7] Achiam J, Held D, Tamar A, Abbeel P. Constrained policy optimization. In: Proceedings of the 34th international conference on machine learning-Volume 70. JMLR. org; 2017, p. 22–31.

[8] Schulman J, Levine S, Abbeel P, Jordan M, Moritz P. Trust region policy optimization. In: International conference on machine learning; 2015. p. 1889–97.

[9] Tessler C, Mankowitz DJ, Mannor S. Reward constrained policy optimization. In: International conference on learning representations. 2019, https://openreview.net/forum?id=SkfrvsA9FX.

[10] Montgomery WH, Levine S. Guided policy search via approximate mirror descent. In: Advances in neural information processing systems. 2016, p. 4008–16.

[11] Levine S, Koltun V. Guided policy search. In: Proceedings of the 30th international conference on machine learning (icml-13); 2013. p. 1–9.

[12] Mei J, Xiao C, Huang R, Schuurmans D, Müller M. On principled entropy exploration in policy optimization. In: Proceedings of the twenty-eighth international joint conference on artificial intelligence, IJCAI-19. International Joint Conferences on Artificial Intelligence Organization; 2019, p. 3130–6. http://dx.doi.org/10.24963/ijcai.2019/434.

[13] Wang Q, Li Y, Xiong J, Zhang T. Divergence-augmented policy optimization. In: Advances in neural information processing systems 32. Curran Associates, Inc.; 2019, p. 6097–108, http://papers.nips.cc/paper/8842-divergence-augmented-policy-optimization.pdf.

[14] Peters J, Mülling K, Altun Y. Relative entropy policy search. In: AAAI. Atlanta; 2010, p. 1607–12.

[15] Miyashita M, Yano S, Kondo T. Mirror descent search and its acceleration. Robot Auton Syst 2018;106:107–16. http://dx.doi.org/10.1016/j.robot.2018.04.009, URL: http://www.sciencedirect.com/science/article/pii/S0921889017307546.

[16] Vieillard N, Pietquin O, Geist M. On connections between constrained optimization and reinforcement learning. In: Optimization foundations for reinforcement learning workshop at NeurIPS; 2019.

[17] Bauschke HH, Hare WL, Moursi WM. A derivative-free comirror algorithm for convex optimization. Optim Methods Softw 2015;30(4):706–26. http://dx.doi.org/10.1080/10556788.2014.968158.

[18] Titov AA, Stonyakin FS, Gasnikov AV, Alkousa MS. Mirror descent and constrained online optimization problems. In: Evtushenko Y, Jaćimović M, Khachay M, Kochetov Y, Malkova V, Posypkin M, editors. Optimization and applications. Cham: Springer International Publishing; 2019, p. 64–78.

[19] Stulp F, Sigaud O. Policy improvement: Between black-box optimization and episodic reinforcement learning. In: Journées francophones planification, décision, et apprentissage pour la conduite de systèmes. France; 2013, p. 1–15, URL: https://hal.archives-ouvertes.fr/hal-00922133.

[20] Deisenroth MP, Neumann G, Peters J, et al. A survey on policy search for robotics. Found Trends Robot 2013;2(1–2):1–142.

[21] Ijspeert AJ, Nakanishi J, Schaal S. Learning attractor landscapes for learning motor primitives. In: Advances in neural information processing systems. 2003, p. 1547–54.